The Bridge Between Neural Toolchains

Vector in, vector out — Jneopallium as the reasoning stage between neural systems.

ADVISORY

— THE PROBLEM

Neural stacks don't live alone

- A neuromorphic or simulation model produces vectors — and then needs something to act on them.
- Gluing toolchains together usually means brittle, bespoke adapters.
- And a demo that needs a full Python runtime installed is a demo nobody runs.

The gap is a clean, typed stage that sits between systems.

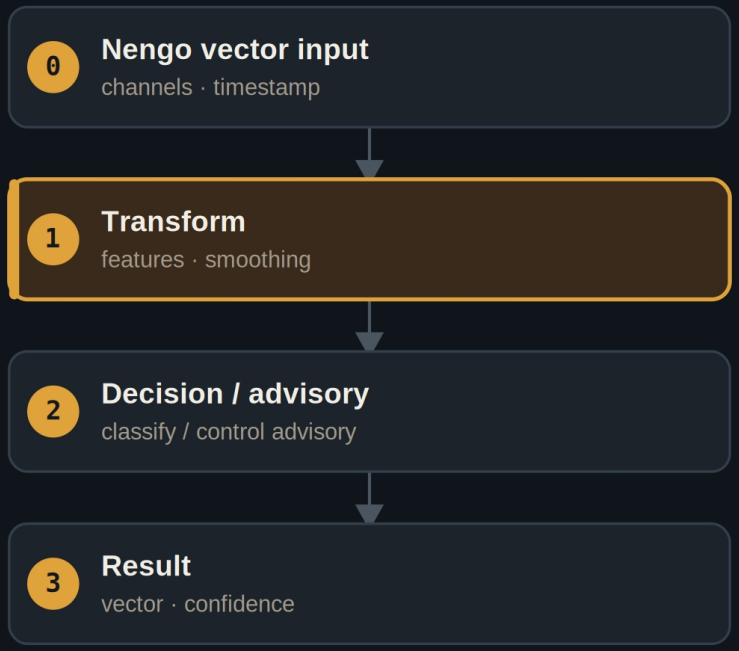
— THE SOLUTION

A reasoning stage with a clean contract

- **It ingests**
  a Nengo-style vector stream — channels and a timestamp.
- **It transforms**
  temporal feature extraction and smoothing, then a decision.
- **It emits**
  an output vector with an explicit confidence.

— HOW IT WORKS

Vectors in, vectors-with-confidence out



A transform layer smooths the stream before the decision layer.

— THE PROOF

A clean interop contract

75

output vectors from the deterministic stream

`rows out`

Confidence

emitted on every output frame

No Python

mock source — runs without a Nengo install

— WHERE IT SITS

A well-behaved middle stage

UPSTREAM

- A Nengo / neuromorphic model produces vectors.
- Mock today; a live feed tomorrow.

DOWNSTREAM

- Receives a vector + confidence.
- Owns whatever happens next.

— THE CEILING

Advisory, by role

ADVISORY Emits a vector and a confidence; the receiving system decides.

Typed contract Vectors in, vectors-with-confidence out — stable across swaps.

No install needed A deterministic mock source proves the contract in CI.

“

Interoperability isn't glamorous — but a clean, typed stage between toolchains is what makes the glamorous parts composable.

The case for Demo 09

Sit between the systems.

A typed vector-in, vector-out reasoning stage with confidence.

github.com/rakovpublic/jneopallium

ADVISORY · deterministic